

과정 [취업기업확대] AI활용 풀스택(프론트엔드, 백엔드) 부트캠프 자바 · 파이썬 · 플러터

기간 2026.07.02 → 2026.07.15

2ND PROJECT

Spring Breeze ERP v2

회사·자원

인사·평가

전자결재

프로젝트·태스크

CONTENTS

목차

PART A

프로젝트 소개

- 01 팀원 소개
- 02 기획 의도
- 03 프로젝트 개요

PART B

시스템 설계

- 04 기술 스택
- 05 요구사항 정의서
- 06 ERD
- 07 시스템 구성도
- 08 스케줄

PART C

개인 파트 · 회고

- 09 최윤정 · 부서 이관
- 10 정수정 · 인사 평가
- 11 김주엽 · 전자 결재
- 12 최다영 · 프로젝트
- 13 각자의 회고 · Q&A

TEAM

팀원 소개 및 담당 파트

각자 하나의 파트를 맡고, 조직 · 인증 · 데이터를 공유했습니다.



TEAM LEAD

최윤정

CHOI · YOON JEONG

회사 · 부서 · 자원 예약

MEMBER

정수정

JEONG · SU JEONG

인사 · 평가 · 권한

MEMBER

김주엽

KIM · JU YEOP

전자 결재

MEMBER

최다영

CHOI · DA YOUNG

프로젝트 · 태스크 · 공지

WHY

SPRING BREEZE-ERP

회사에서 실제로 자주 쓰는 시스템,
많은 도메인이 얹히는 구조를 통해서 배우자.

01

REAL

실무형 도메인

결재 · 인사 · 자원 · 프로젝트 — 회사에 들어가면
매일 만나는 시스템

02

COMPLEX

구조 학습

4개 파트가 사원 · 부서를 공유하는 구조를 처음부터
협업으로 설계

03

FULLSTACK

풀스택 + AI 통합

백엔드 · DB · 뷰 · AI · 외부 API를 하나의 서비스에
모두 다룰 수 있음

PROJECT OVERVIEW

프로젝트 개요

전사 통합 ERP를 4명이 7일 동안 다시 지었습니다.

TEAM

4

 MEMBERS

팀 구성

DURATION

14

 DAYS

개발 기간

MODULES

6

 DOMAINS

핵심 파트

NEW

8+

 FEATURES

신규 기능

SUMMARY

1차 프로젝트의 ERP를 Spring Boot · Thymeleaf · Oracle · AI로 재구성

TECH STACK

기술 스택

개발 스택

BACKEND



Java
17



Spring Boot
3.x



Spring Security
6



MyBatis
XML

FRONTEND



Thymeleaf
SSR



Bootstrap
5



JavaScript
Vanilla

DATABASE



Oracle
c XE

EXTERNAL API



OpenAI
GPT-4o-mini



Discord
Webhook



Naver OCR



SMTP
Mail

협업 도구



GitHub
영상 관리



Discord
실시간 소통



Notion
문서화

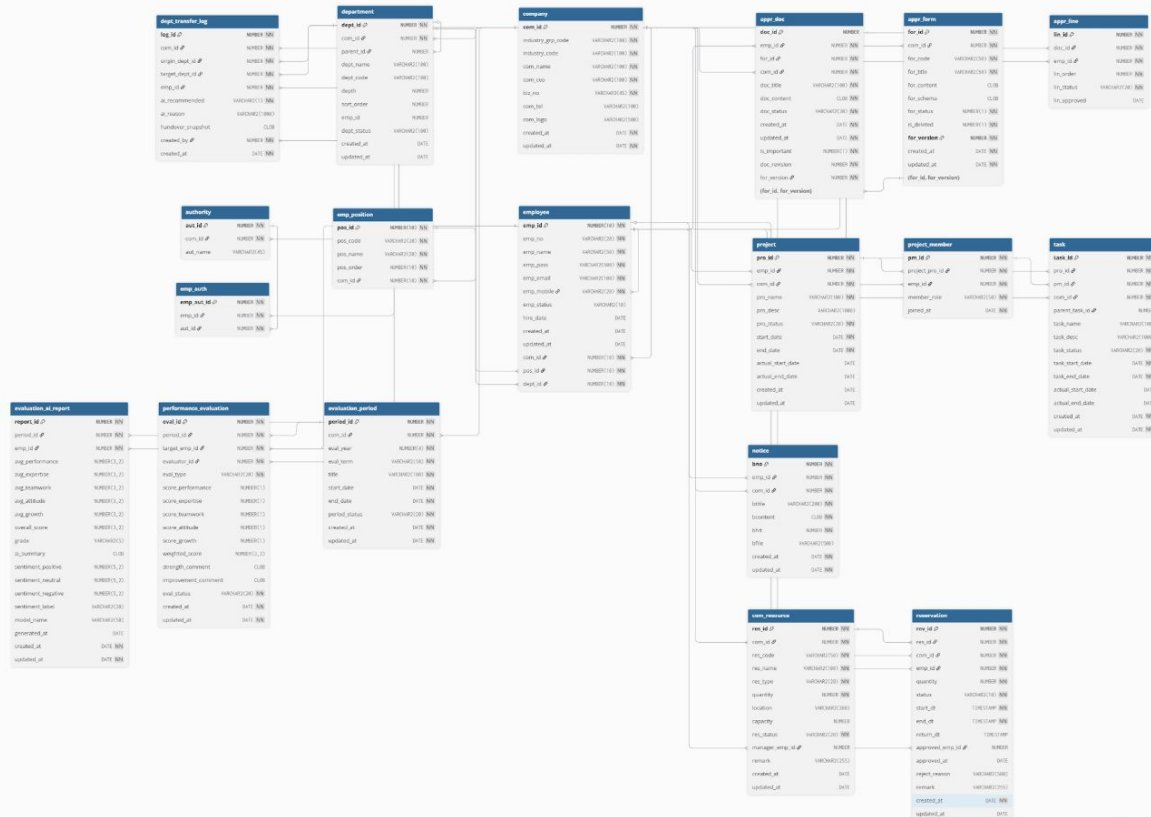
REQUIREMENTS

요구사항
정의서

1-2	결재 문서	양식 관리	양식 조회	양식 조회	조회 페이지 list_form/detail_form	양식 조회	> 회사 설정화 여부, 코드를 이용해서 양식 리스트 검색 detail_form > 기존에 작성했던 양식의 상세 정보 표시 detail_form 또는 list_form 에서 상세페이지로 이동하여 기존 양식 수정하여 저장 1. 페이지를 불러와 for_content_for_schema를 기준으로 num값의 유무로 어느페이지인지 제어하여 동적쿼리문본을 나눔 2. 양식 내용(이름)을 기준으로 수정할때 원래 본문과 대조하여 for_version을 불러거나 유지성지 (1-6 유지성지 제외) 3. 1-5 파트의 LLM이 만들어준 양식의 데이터 JSON를 기존 양식 본문 파트에 해석하여 동적으로 생성 (2-6) 이때와 JSON과 필드값을 제거/수정하여 저장할때 다시 JSON타입으로 바꾸어 필드값을 반영시킴	1순위	TRUE	TRUE	TRUE	
1-3	결재 문서	양식 관리	양식 수정	양식 수정	수정 페이지 update_form	양식 수정	사용자가 원하는 정보를 기반으로 is_deleted를 새로 추가하여 삭제된 양식인지 여부를 확인 양식 수정 / 양식이 삭제 될 경우 데이터를 지워버리는게 아닌 is_deleted를 새로 추가하여 삭제된 양식인지 여부를 확인 양식 수정 / 양식이 수정됐을때 for_title_for_content_for_schema 세가지를 대조하여 한가지라도 기준값과 다를경우 for_version의 수치를 불러 양식 수정과 결재 문서 작성에 동시에 이루어질때 데이터 고입 방지	3순위	TRUE	TRUE	TRUE	미완화 부분 1-5 LLM 사용 파트 작성하면서 잘이 조정된 예정
1-4	결재 문서	양식 관리	양식 제거	양식 제거	수정 페이지내 버튼	양식 삭제	update_form에서 하단 삭제 버튼 누를시 삭제여부 재확인 후 처리	4순위	TRUE	TRUE	TRUE	
1-5	결재 문서	양식 관리	양식 생성	양식 생성	양식 작성 페이지 write_form	양식 생성	사용자가 원하는 정보를 기반으로 is_deleted를 새로 추가하여 삭제된 양식인지 여부를 확인 양식 수정 / 양식이 수정됐을때 for_title_for_content_for_schema 세가지를 대조하여 한가지라도 기준값과 다를경우 for_version의 수치를 불러 양식 수정과 결재 문서 작성에 동시에 이루어질때 데이터 고입 방지	3순위	TRUE	TRUE	TRUE	
1-6	결재 문서	양식 관리	동시성 제어	양식 수정 / 양식 제거	양식 수정 페이지 update_form	양식 동시성 제어	양식 수정 / 양식이 수정됐을때 for_title_for_content_for_schema 세가지를 대조하여 한가지라도 기준값과 다를경우 for_version의 수치를 불러 양식 수정과 결재 문서 작성에 동시에 이루어질때 데이터 고입 방지	1순위	X	TRUE	TRUE	
2-1	결재 문서	문서 관리	문서 조회	문서 조회 / 사용자 문서 조회 / 관리자	조회 페이지 / list_doc 조회 페이지 / 미팅	결재 문서 조회	내가 결재해야한 문서 / 모든 문서 조회	2순위	TRUE	TRUE	TRUE	
2-2	결재 문서	문서 관리	문서 작성	문서 작성	작성 페이지 write_doc	결재 문서 작성	list_form과 유사하게 최소 1가지 이상의 조건으로 검색 작성 페이지로 이동할때 수정 불가능한 값들은 전부 가져와 대입 양식 파트에서 만든것인 결재 양식 문서본문 데이터를 가져와 사용자가 선택한 양식의 데이터를 summernote에 종류 문서 체크시 결재는 1항에서 3항으로 늘어나게 설계 list_doc에서 이어지는 화면으로, 문안이 결재해야 하는 문서 혹은 아직 미승인,반려 상태 수정이 가능한 문서 일경우 해당 결재 문서를 보고 수정, 혹은 반려 승인등 처리할수있음	2순위	FALSE	FALSE	FALSE	3차례 예정
2-3	결재 문서	문서 관리	문서 처리	문서 수정 / 사용자	상세 페이지 detail_doc	결재 문서 결재	내가 결재해야한 문서 / 모든 문서 조회	4순위	TRUE	TRUE	TRUE	
2-4	결재 문서	문서 관리	동시성 제어	문서 수정	상세 페이지에서 승인/반려서 처리	결재 문서 동시성 제어	내가 결재해야한 문서 / 모든 문서 조회	2순위	X	TRUE	TRUE	결재 승인/반려 전 수정할수 있는 페이지 7번 이후에 그부분도 따로 테스트 해봐야함
2-5	결재 문서	문서 관리	문서 수정	문서 수정	수정 페이지 update_form	결재 문서 수정	내가 결재해야한 문서 / 모든 문서 조회	5순위	FALSE	FALSE	FALSE	어느정도 로직은 구현 완료된것같은데 수정 페이지 3차례 만들어서 검증해야함 완전완료
2-6	결재 문서	문서 관리	문서 작성	문서 작성	작성 페이지 write_doc	Dynamic UI 렌더링	내가 결재해야한 문서 / 모든 문서 조회	3순위	TRUE	TRUE	TRUE	
3-1	결재신	결재신 관리	결재신 생성	결재신 생성	2-2 결재문서 작성 페이지 / write_doc	결재신 저장	내가 결재해야한 문서 / 모든 문서 조회	3순위	TRUE	TRUE	TRUE	
3-2	결재신	결재신 관리	결재신 조회	결재신 조회	2-3 결재 문서 조회 페이지 / update_doc	결재신 조회	내가 결재해야한 문서 / 모든 문서 조회	4순위	TRUE	TRUE	TRUE	
3-3	결재신	결재신 관리	결재신 수정	결재신 수정	2-3 결재 문서 조회 페이지	결재신 수정	내가 결재해야한 문서 / 모든 문서 조회	6순위	FALSE	FALSE	FALSE	

ERD

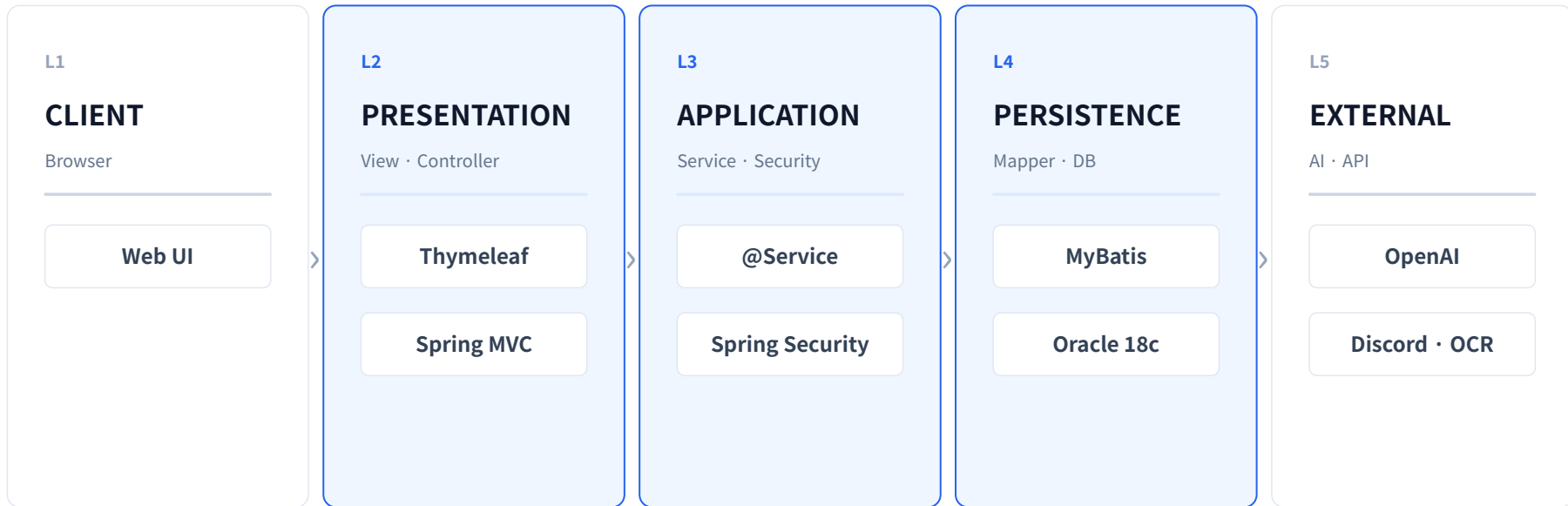
ERD



SYSTEM FLOW

시스템 구성도

요청이 이렇게 진행됩니다 · L1 → L2 → L3 → L4 → L5



SCHEDULE

7.2 → 7.15, 14일간의 여정

리팩토링 → 신규 기능 → QA & 시연. 두 개의 마일스톤으로 진행률을 관리했습니다.



개인 파트

AGENDA

이제부터 이 순서로 진행됩니다

01



TEAM LEAD

최윤정

회사 · 부서 · 자원 예약



02



MEMBER

정수정

인사 · 평가 · 권한



03



MEMBER

김주엽

전자 결재



04



MEMBER

최다영

프로젝트 · 태스크 · 공지



PART 01

회사 · 부서 · 자원 예약

TEAM LEAD

최윤정

PAIN

업무 중에 이런 게 불편했다

아우 귀찮아, 이걸 언제 일일이 다 작성하고 있어?

엥? 사업자 번호 형식이 맞는데 실제론 없다고?

이 사람이 어떤 부서로 이동하는지 어떻게 알아 내가?

뭐야, 너도? 나도? 그래서 이거 누구кен데?

AI · KEY WORDS

이렇게 AI를 활용했다

멀티파트 요청(message+file)
OCR 인식 실패 방어Connect/Read Timeout
장애 시 Fallback 응답 설계실시간 잔여수량 조회
기간 겹침(Overlap) 검증FK 구조 기반 이관 범위 축소
@Transactional 전체 롤백

“한 장이면 끝, 타이핑은 없다”

업종 매핑 8종 + INDUSTRY_MAP 텍스트-코드 매핑, 매칭 실패 시 사용자 직접 입력으로 100% 전환 보장

1/5

Naver OCR

⑤ 사업자등록증 자동입력

이미지 업로드

파일 선택 선택된 파일 없음

JPG · PNG · PDF

취소 ✓ 폼에 적용

⑤ 사업자등록증 자동입력

이미지 업로드

파일 선택 bizno_license.png

JPG · PNG · PDF

사업자등록번호 368-88-03013

회사명 (주)글로벌데이터로드

대표자 이상욱

개업일자 2024-06-24

업종 (OCR 인식 텍스트로 자동 매핑 · 확인란 적절 선택)

정보통신업

세부 업종 (OCR 인식 텍스트로 자동 매핑 · 확인란 적절 선택)

58222 · 응용 소프트웨어 개발 및 공급업

취소 ✓ 폼에 적용

6개 필드 자동 인식

사업자등록증 이미지 1장 →

사업자번호 · 상호명 · 대표자명 · 개업일자 · 업종 · 세부업종

끊김 없는 등록 플로우

인식 실패 시 직접 입력 전환

등록 소요시간

▼ 약 69.57%

[Before] 46s → [After] 14s

“있는 회사인지, 존재하는지부터”

국세청 API 응답 지연 시에도 대응(connect 3초 · read 5초 상한), 초과 시 타임아웃 전용 Fallback 응답으로 서버 스텔드 방어

2/5

data.go.kr

The image displays three overlapping screenshots of the '기본 정보' (Basic Information) section of a company registration form on data.go.kr. The form fields include: 회사명 (Company Name), 대표자 (Representative), 사업자등록번호 (Business Registration Number), 대표 전화 (Representative Phone), 개업일자 (Opening Date), and a checkbox for '국세청 진위확인' (Korea Tax Authority Verification).
- The top-left screenshot shows a successful state where the '국세청 진위확인' checkbox is checked and green, with a message: '진위확인 시에만 사용되며 DB에는 저장되지 않습니다. ① 등록하려면 진위확인이 필요합니다.' (Only used when verified, not stored in DB. ① Registration requires verification).
- The bottom-left screenshot shows an error state where the '사업자등록번호' field has a red border and a red error message: '확인할 수 없습니다. 형식: 123-45-67890' (Cannot verify. Format: 123-45-67890).
- The middle-right screenshot shows a loading state where the '국세청 진위확인' checkbox is disabled with a grey border and a message: '국세청에 확인 중...' (Verifying with Korea Tax Authority...).

실시간 3중 검증

국세청 공공데이터 API →

사업자번호 · 대표자명 · 개업일자 · 휴폐업 여부 검증

검증 미통과시 화면단에서 등록 자체 차단

정보가 변경된 경우도 고려, 진위 확인을 재점검 유도

잘못된 사업자번호 유입 차단

사업자 등록형식만 맞는 경우 차단 **100%**

“AI는 추천만, 결정은 사람이”

이관 후보 부서가 하나도 없는 경우(신설 부서 등)에도 **전체 활성화 부서로 자동 풀백**해, 관리자가 선택지 자체가 없어 이관을 못 하는 상황을 방지

자원예약, 결재대기 라인, 진행중인 결재문서 고려

물류팀 DPT00020 부서는 현재 이관 대기(PENDING_DELETE) 상태입니다. 아래 소속 사원을 모두 다른 부서로 이관해야 부서 식체가 최종 완료됩니다.

AI 추천 부서: 생산부

남은 결재 문서는 생산 및 현장 운영과 밀접하게 연관된 내용이 많아 생산부가 이관 부서로 적합합니다. 생산부에서 관련 업무를 효율적으로 이룰 수 있습니다.

이 주전을 전체 사원에 적용

소속 사원 17 명

미처리 예약 6 건

결재 대기(라인) 10 건

상신 진행중 9 건

① 예약 할계 항목은 emp_id 기준으로 사원을 따라가는 구조라 별도 이관 작업 없이 사원의 소속 부서만 변경하면 자동으로 할계 이동합니다. 아래 목록은 참고용입니다.

소속 사원 이관

NO	사원	이름	직급	이관할 부서
1	ERP-000160	공시영	사원	이관할 부서 선택
2	ERP-000065	이도준	사원	이관할 부서 선택
3	ERP-000028	채상열	사원	이관할 부서 선택
4	ERP-000098	홍은우	사원	이관할 부서 선택
5	ERP-000293	황대진	사원	이관할 부서 선택
6	ERP-000124	고상현	주임	이관할 부서 선택
7	ERP-000289	박건빈	주임	이관할 부서 선택
8	ERP-000127	방호진	주임	이관할 부서 선택

이관 취소 (부서 필적) 이관 완료

이관할 부서 선택

부서	이름	직급	이관할 부서
생산부	공시영	사원	이관할 부서 선택
생산부	이도준	사원	이관할 부서 선택
생산부	채상열	사원	이관할 부서 선택
생산부	홍은우	사원	이관할 부서 선택
생산부	황대진	사원	이관할 부서 선택
생산부	고상현	주임	이관할 부서 선택
생산부	박건빈	주임	이관할 부서 선택
생산부	방호진	주임	이관할 부서 선택

이관할 부서 선택

부서	이름	직급	이관할 부서
생산부	공시영	사원	이관할 부서 선택
생산부	이도준	사원	이관할 부서 선택
생산부	채상열	사원	이관할 부서 선택
생산부	홍은우	사원	이관할 부서 선택
생산부	황대진	사원	이관할 부서 선택
생산부	고상현	주임	이관할 부서 선택
생산부	박건빈	주임	이관할 부서 선택
생산부	방호진	주임	이관할 부서 선택

AI가 유사 부서를 추천

부서 + 사유로 제시하고
관리자가 "전체 적용" 버튼으로 일괄 반영

한 명이라도 실패하면 전체 롤백

전원의 이관 부서가 채워지기 전에는 완료 비활성화
이관 이력 화면에서 AI 추천 수용 여부(Y/N) 감사 기록

잘못된 부서이관 처리 방지

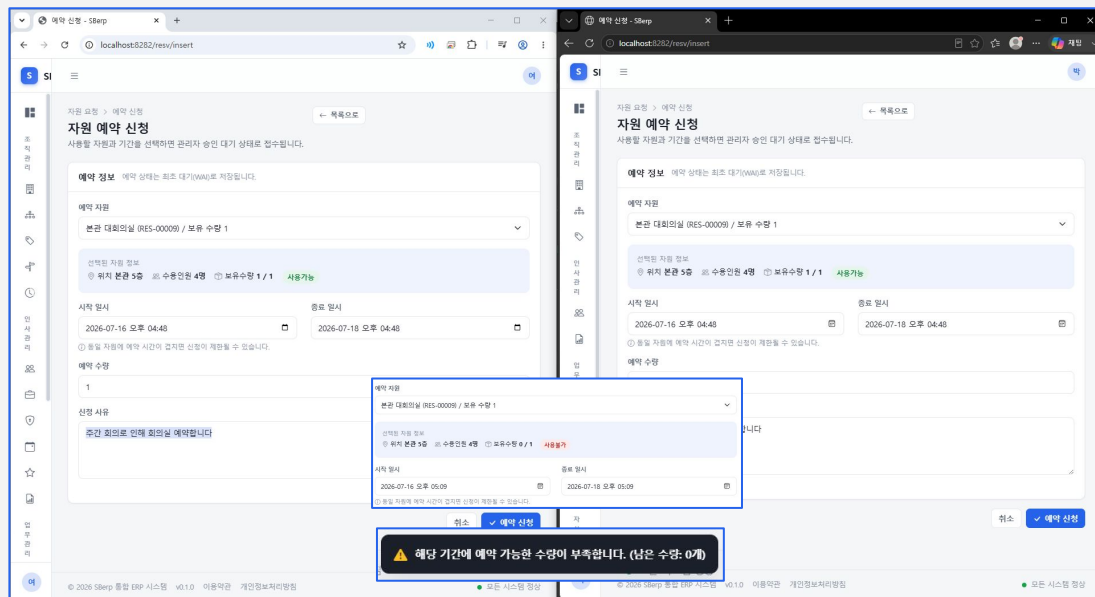
트랜잭션 실패 시 데이터 정합성 유지율 **100%**

“마지막 1개, 누가 먼저일까”

res_id, start_dt, end_dt 복합 인덱스로 기간 겹침 조회 성능 확보

4/5

동시 예약 접근



총 보유수량 - 대기/승인된 예약수량

자원·기간 선택 시 실시간 잔여수량 조회

동일 기간 겹침 검증

신청 시점에 서버가 동일 기간 겹침까지 다시 계산

자원 부족 시 등록 자체 차단

redirect + 토스트 + 필드 하이라이트

부족 시 사유(남은 수량: N개)와 함께 안내

동시 진행 후 한쪽 예약 신청 불가 메시지 출력

“부서를 지운다는 건, 함께 옮긴다는 것”

부서 삭제 마법사

C

CONTEXT

문제 상황

부서를 삭제하려면 소속 사원을 다른 부서로 옮겨야 하는데, 관리자가 조직 전체 상황을 다 알기 어려워 어디로 옮길지 판단하기 어려웠습니다.

A

ACTION

분석 · 결정

예약·결재는 부서가 아닌 사원을 참조하는 구조라는 걸 파악해 이관 대상 사원 하나로 좁혔고, 여러 사원 이관을 **@Transactional**로 묶어 **하나라도 실패하면 전체 롤백**되도록 설계했습니다.

R

RESULT

결과

dept_status 3단계 관리
(**ACTIVE→PENDING_DELETE→DELETED**)

에러 코드 기반 예외 처리로 실패 사유 구분, 사원 전원 이관 완료 시 부서 자동 소프트 삭제.

L

LEARNING

성장

처음엔 예약·결재까지 각각 다른 테이블에서 옮겨야 한다고 생각해 로직이 복잡해질 걸 걱정했는데, 둘 다 **emp_id**를 참조하고 있었습니다. 덕분에 사원의 소속 부서 하나만 바꾸면 나머지는 **FK**를 타고 자연스럽게 따라온다는 걸 알게 됐고, 불필요한 이관 로직을 미리 걸어버렸습니다.

PART 02

인사 · 평가 · 권한

MEMBER

정수정

PAIN

업무 중에 이런 게 불편했다

신입 사원이 몇 명인데 똑같은 안내 메일을 계속 보내요?

전 사원의 인사 평가를 저 혼자 확인해서 보고하라고요?

AI · KEY WORDS

이렇게 자동화하고 AI를 붙였다

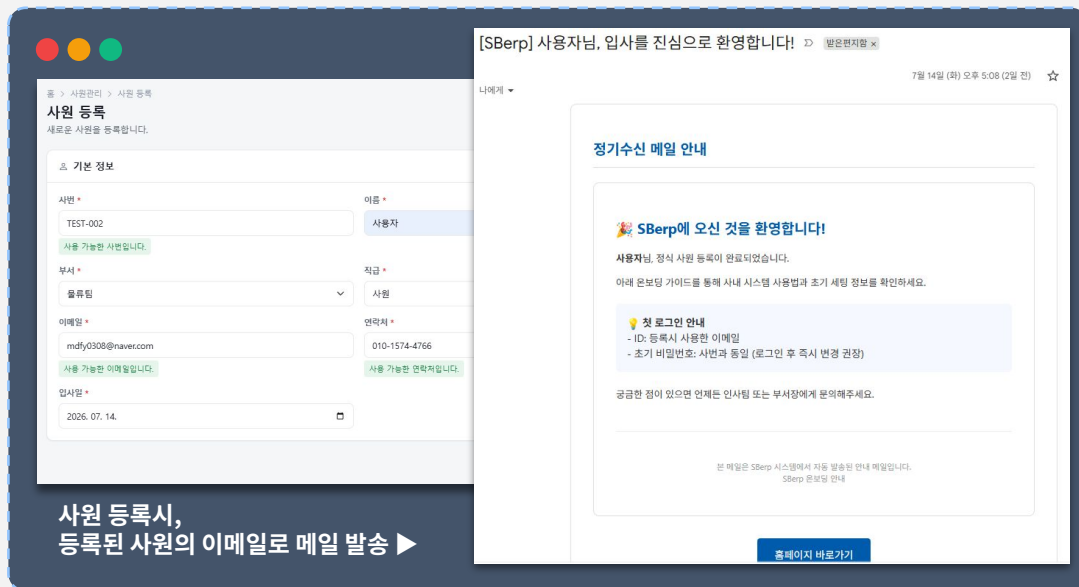
온보딩 메일 자동 발송

평가 코멘트 AI 요약

Mock fallback
응답 안전 장치

“등록 이후, 관리는 시스템에 맡기세요!”

사원을 등록하면 온보딩 메일이 바로, 3일 뒤에는 적응 확인 메일이 자동으로 나갑니다.



1/3
 사원 온보딩 이메일 자동화

온보딩 메일 발송

즉시

사원 정보가 저장되는 순간 자동으로 나갑니다.

적응 확인 메일

D+3

입사 3일 뒤에 시스템이 알아서 보냅니다

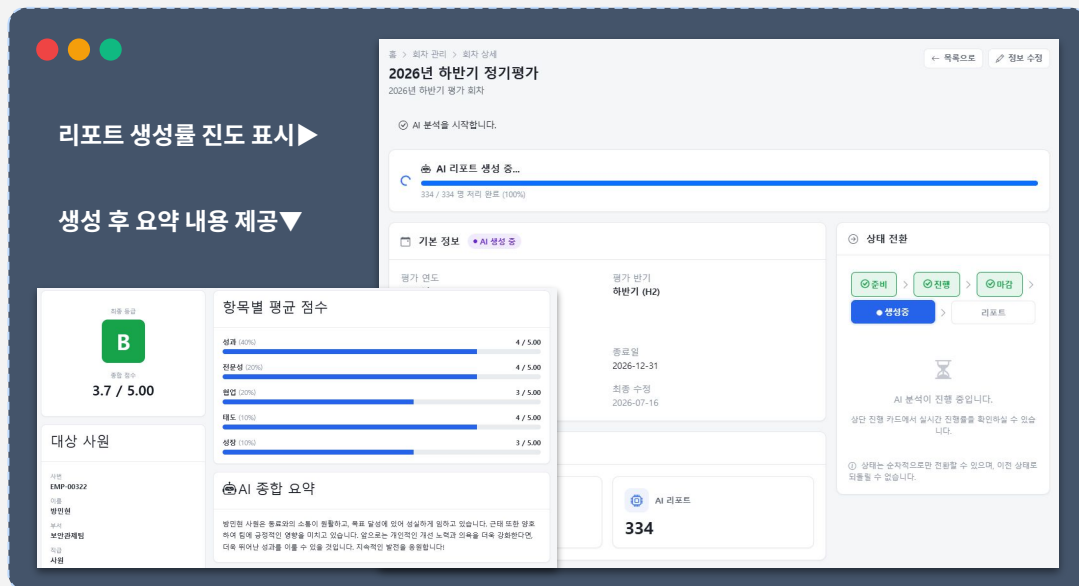
안전 발송 장치

발송 실패 0 건

저장이 실패한 사원에게는 메일이 나가지 않고, 놓친 것은 주기적으로 재확인해 다시 발송합니다.

“요약은 AI가, 신뢰는 로직에서”

평가가 마감되면 AI가 사원별 코멘트를 정리, 도중에 문제가 생겨도 어디서 멈췄는지 바로 보입니다.



2/3 openAI 리포트 자동 요약

평가 회차 진행 단계

6 단계

준비 → 진행 → 마감 → 리포트 생성 → 완료 (실패 시 재시도)

AI 결과

합산 100 %

AI가 이상한 답을 주면, 안전한 기본 결과로 자동 대체합니다

업무를 동시에

백그라운드 실행

리포트를 만드는 동안에도 다른 업무를 그대로 볼 수 있습니다

"AI 리포트 생성이 갑작스럽게 중단 된다면?"

C

CONTEXT

문제 상황

개발 초기에 겪었던 문제 상황. 회차 마감을 누르고 AI가 리포트를 만드는 도중에, 만약 서버가 재시작되거나 응답이 없으면? 처음 이런 상황을 겪었을 때는 실패 단계를 따로 구분하지 않아, 회차가 '완료' 상태인 채로 멈추는 상황이 발생

A

ACTION

분석 · 결정

그래서 이 문제를 해결하고자 회차 진행 단계를 6단계로 세분화해 '실패와 리포트 작업중' 상태를 따로 만들고, 생성 작업에 문제가 생기면 반드시 '실패' 상태로 남도록 처리함.

R

RESULT

결과

결과적으로 실패한 상태가 눈에 보이니까, 관리자 입장에서는 상황을 확인하고 재생성을 시도할 수 있게 됨. 이와 비슷한 문제가 있었던 온보딩 메일 자동 발송에도 같은 방식을 적용해서, 누락된 상태를 확인하고 메일이 다시 발송되도록 문제 해결.

L

LEARNING

성장

이번 과정을 통해서 알게된 건, 잘 돌아가는 경우만을 상정하고 기능을 설계하면 실패했을 때는 문제를 해결하기가 어렵다는 것. 오히려 실패 단계를 눈에 보이게 만드는 것이, 시스템을 원활히 돌아가게 할 수 있다는 것을 배우게 된 과정이었음.

PART 03

전자 결재

MEMBER

김주엽

PAIN

업무 중에 이런 게 불편했다

새로운 결재 양식을 작성할 때마다 요구조건에 맞는 양식을 번거롭게 생성

이미 사용된 결재 양식을 수정 및 삭제한다면
그 양식으로 만들어진 과거 문서들은 어떻게 되는가?

AI · KEY WORDS

이렇게 AI를 활용했다

관리자가 원하는
양식을 문장으로 설명

GPT JSON mode로
구조화된 필드 생성

파싱된 필드를 텍스트/
날짜/셀렉트로 렌더링

"결재 양식 고민은 AI에게 맡기고, '승인' 버튼만 누르세요!"

양식을 수정하거나 삭제해도, 과거에 작성된 문서는 그대로 보존됩니다.

과거 문서 영향

0 건

for_id + for_version 복합키로 버전 분리
과거 문서는 작성 당시 버전을 FK로 계속 참조

삭제 양식 참조

정상 유지

is_deleted 플래그 처리
하드 삭제 대신 목록에서만 제외해 FK 무결성 보존

AI 양식 초안

1 문장 → JSON

GPT JSON mode 강제 · 관리자 검수 후 등록

2/4

"양식 고르면 입력창이 짤! AI가 만들어주는 맞춤형 결재 화면" 동적 스키마

양식 종류에 따라 문서 작성 화면이 실시간으로 달라집니다.

문서 내용
문서 내용

휴가신청서

휴가 종류	(연차 / 오전반차 / 오후반차 / 공개 / 병가) 중 기입	비상 연락처	010-
기간	2026년 __월 __일 ~ 2026년 __월 __일 (총 __일간)		
※ 구체적인 휴가 사유를 입력해 주세요.			
휴가 사유			

위와 같이 휴가를 신청하시니 승인하여 주시기 바랍니다.

신청자 이름 *

신장자 이름 *

테스트

시작 날짜 *

연도-월-일

종료 날짜 *

연도-월-일

휴가 사유

신청자 이름 *

시작 날짜 *

2026-07-16

종료 날짜 *

연도-월-일

휴가 사유

취소 접수함

양식 선택 즉시

필드 자동 생성

forSchema JSON을 파싱해
텍스트/날짜/셀렉트 입력 폼으로 실시간 렌더링

입력 방식
복수 요구

2 가지 통합

에디터 직접 작성과 AI 생성 스키마,
제출 시 동일하게 HTML로 변환해 저장

필드 검증

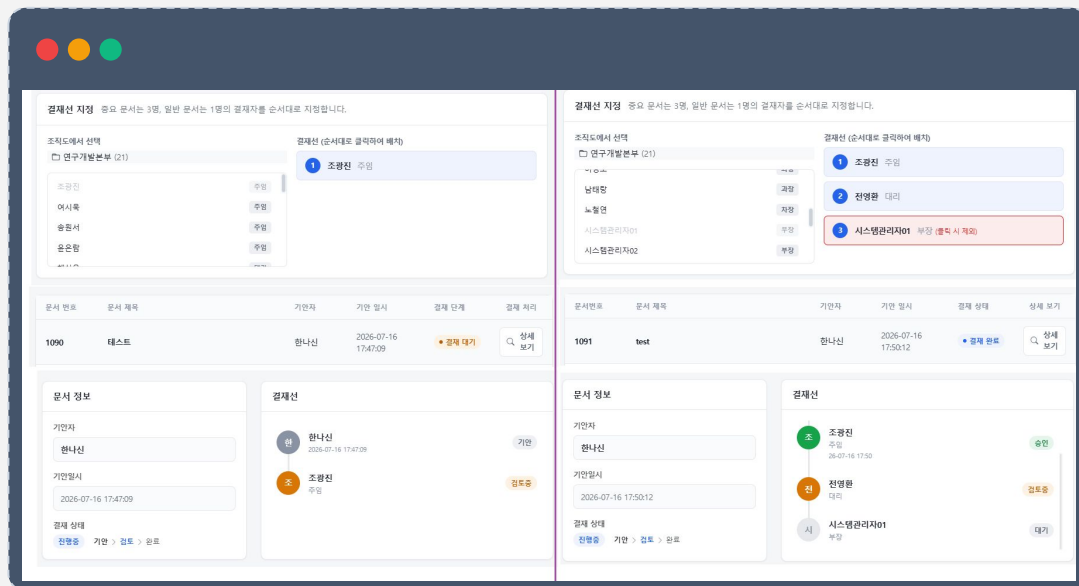
프론트엔드 실시간 체크

required 필드는 제출 전 JS에서 빈 값 검사,
미입력 시 제출 차단

"결재 순서 새치기 불가! 철저한 마이웨이 순차 승인 시스템"

3/4 순차 승인

선택 가능한 결재자만 자동으로 걸러지고, 동시 승인은 구조적으로 막힙니다.



동시 승인 방어

구조적 차단

lin_status(NOT/WAI) 순차 전이 구조로,
동시에 두 명이 결재 대기 상태가 될 수 없음

결재 단계

1명 / 3명

일반 문서는 1명, 중요 문서는 3명 슬롯으로 자동 전환

자동 제외 처리

본인 · 동급자

직급 순서(pos_order) 비교로 상급자만 노출,
클릭한 순서 그대로 결재선에 반영

"양식을 고쳤는데, 버전이 안 올라갔다"

C

CONTEXT

문제 상황

이미 사용된 결재 양식을
수정 및 삭제 해도 과거문서가
안전해야 한다는 목표로,
버전 분리(forVersion)와 소프트 삭제
(isDeleted)를 함께 설계함.
그런데 AI 스키마 기능을 추가 후,
스키마 변경은 감지하지 못해
버전이 그대로인 현상이 뒤늦게 발견

A

ACTION

분석 · 결정

버전 분리 조건이
forContent/forTitle만 비교하고 있었음
AI 스키마 양식은 forContent가 원래
비어있어 변경 없음으로 읽히는
구조였던것. forSchema 비교를
추가하고, 기존 필드처럼
공백 차이로 인한 오탐을 막기 위해
동일한 방식으로 비교

R

RESULT

결과

에디터/AI 스키마 두 방식 모두 내용이
바뀌면 예외 없이 새 버전으로 분리됨을
여러 차례 직접 수정하며 확인.
과거 버전은 그대로 보존돼 기존문서의
참조 무결성도 안전하게 유지

L

LEARNING

성장

새 기능을 추가할 때, 이 기능이 건드리는
값이 기존 비교/검증 로직에도
반영됐는지를 매번 확인해봐야 한다는걸
체감하였고, 내가 추가한 기능이
어떤 코드들에 영향이 가고
코드의 시작점과 도착점까지 다시
체크해봐야 한다는게 중요하다는걸
깨달았음

PART 04

프로젝트 · 태스크 · 공지

MEMBER

최다영

PAIN

업무 중에 이런 게 불편했다

일정 하나 고치면, 나머지는 나 몰라라

위험한 프로젝트, 태스크를 하나하나 다 열어봐야 합니다

팀장은 취합만, 개발자는 정리만 하다 하루 감

AI · KEY WORDS

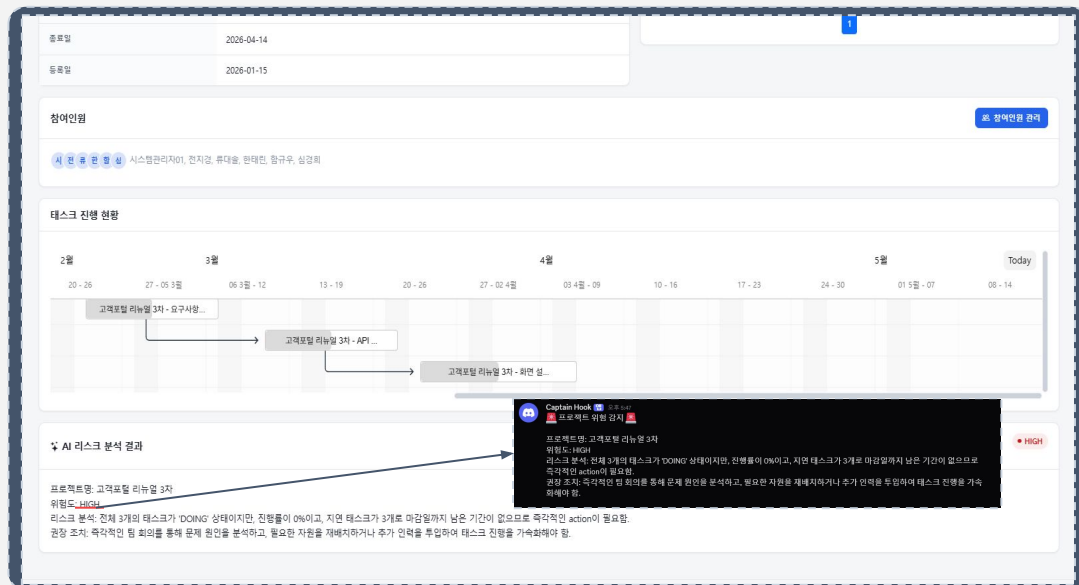
이렇게 AI를 활용했다

재귀 CTE
순환참조 방지리스크 판정
Discord 실시간 알림Google Docs
역할별 분기 보고서

"한눈에 보이는 트리, 미리 짚히는 위험"

선/후행 태스크를 SQL 한 번에 조회. HIGH 리스크는 Discord로 자동 알립니다.

Discord Webhook



태스크 트리 깊이

제한 없음

Oracle 재귀 CTE — 트리 깊이 상관없이 한 번에 조회

일정 갱신 쿼리

1회

후행 태스크까지 벌크 업데이트

AI 리스크 판정

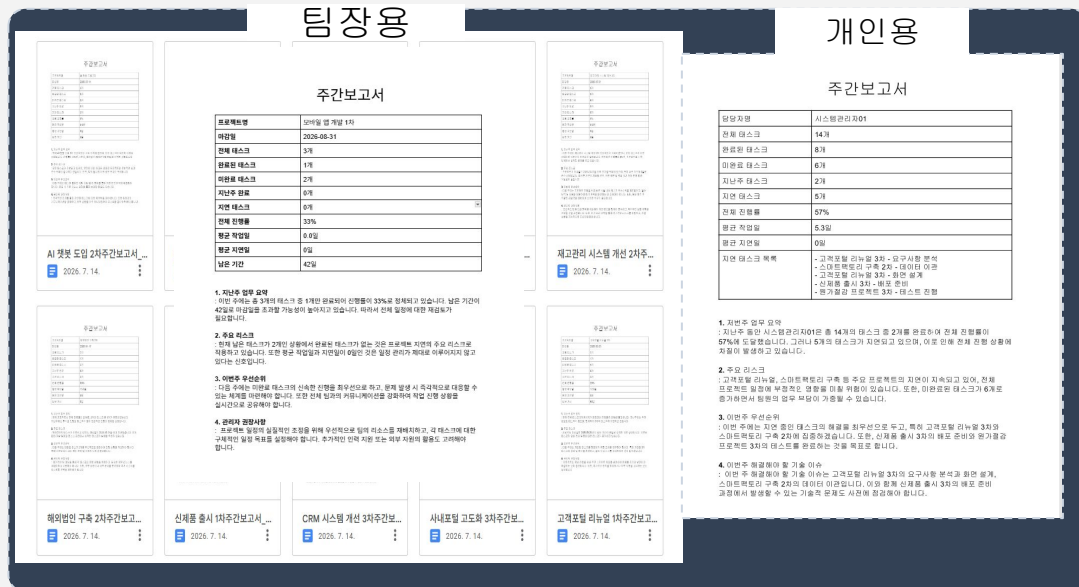
3 단계

정량적 지표 종합->ai 리스크 분석+요약
HIGH · MID · LOW · / HIGH 시 Discord Webhook

"보고서 취합은 AI가, 저는 클릭만 합니다"

프로젝트 데이터를 SQL로 집계하고 AI가 요약해, 관리자용은 Google Docs로 자동 저장하고 개인용은 PDF로 즉시 받습니다.

2/3
주간 보고서 자동화



팀장용 자동 생성(@Scheduled) + 활성 프로젝트 전체 순회

매주 월

프로젝트 전체 순회 → 정량 데이터 집계 → AI 분석/요약

산출물

2종

팀장용-Google drive 자동 저장
개인용-버튼 클릭 시 PDF 즉시 다운로드

역할별 AI 프롬프트 분기

2종

팀장용(거시 일정 리스크) / 개인용(본인 담당 기술이슈)
→ 다른 관점의 프롬프트로 분리 생성

"선행 일정이 바뀌었는데 후행 일정은 그대로라면?"

C

CONTEXT

문제 상황

태스크 일정을 수정하면 후행(자식) 태스크가 연쇄적으로 밀려나야 하는데, 자바 for 루프로 한 단계씩 밀다 보니 손자·증손자 태스크는 반영되지 않고, 매 수정마다 프로젝트 전체 태스크를 재조회하는 비효율까지 겹침.

A

ACTION

분석 · 결정

Oracle 재귀 CTE(START WITH ... CONNECT BY PRIOR)로 후행 태스크 트리를 한 번의 쿼리로 조회, PL/SQL 익명 블록으로 벌크 업데이트. 순환 참조는 ck_task_not_self_parent CHECK + 자바 isCyclic() 이중 방어.

R

RESULT

결과

5단계 이상 깊이의 의존성 트리도 한 번의 API 호출로 정확히 갱신. 반응 속도 체감상 크게 개선.

L

LEARNING

성장

계층·트리 데이터는 자바 재귀보다 SQL 재귀 CTE가 훨씬 자연스럽게 빠르다는 것을 배움. '루프로 밀어내기'에서 '한 번에 계산하기'로 사고 전환의 순간.

TEAM RETROSPECTIVE

각자의 회고

이 프로젝트에서 우리가 발견한 것, 다음에 다르게 할 것.

“

TEAM LEAD

최윤정

회사 · 부서 · 자원 예약

단순 CRUD인 줄 알았던 삭제 기능 하나에도 많은 안전장치가 필요하다는 걸 체감했고, 단순 DELETE 쿼리 하나 짤 줄 알았는데 FK 걸린 데이터 처리하려니 트랜잭션 묶고 예외 케이스 챙기는 게 생각보다 컸습니다.

“

MEMBER

정수정

인사 · 평가 · 권한

기술 자체의 적용보다 실제로 사용자에게 필요한 기능이 무엇인지 고려하는 것이 어려운 일이었습니다. 메일 발송이든 리포트 요약이든, 기능 개발은 반복되는 사용자의 노력과 시간을 절약하는 방향으로 설계하는 것이 중요하다고 느꼈습니다.

“

MEMBER

김주엽

전자 결재

양식 수정/삭제 시 기존 문서에 미칠 영향을 인지했고, 삭제 시에는 논리 삭제를, 수정 시 신규 버전 생성 방식을 구현했습니다. 이 과정을 통해 기능 하나를 개발할 때 전체 데이터에 영향이 가게 되는 걸 먼저 생각하는 습관이 생겼습니다.

“

MEMBER

최다영

프로젝트 · 태스크 · 공지

태스크 의존성과 일정 연쇄 변경 기능을 구현하면서 하나의 기능이 다른 데이터에 미치는 영향을 고려하는 경험을 할 수 있었습니다. 또한 권한 처리와 예외 상황을 함께 설계하며 안정적인 서비스를 만드는 과정의 중요성을 배울 수 있었습니다.

감사합니다

Spring Breeze ERP v2

· 2ND PROJECT ·

Q & A

궁금하신 점 있으세요?

윤

최윤정

회사 · 자원

수

정수정

인사 · 평가

주

김주엽

전자 결재

다

최다영

프로젝트 · 공지